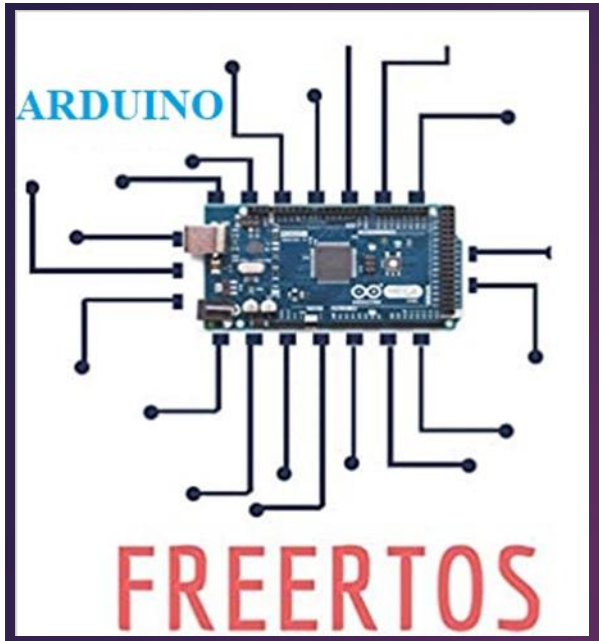


Chương 2: (tiếp)
FreeRTOS – Task Synchronization & Inter-Task Communication

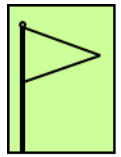
Nội dung

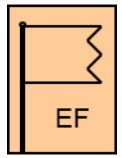
- Đồng bộ tác vụ
- Truyền thông liên tác vụ

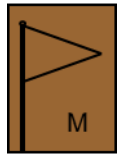
ĐỒNG BỘ TÁC VỤ

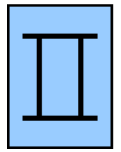
- **Giới thiệu về đồng bộ tác vụ**
- Semaphore trong FreeRTOS
- Mutex trong FreeRTOS

Tại sao cần đồng bộ?









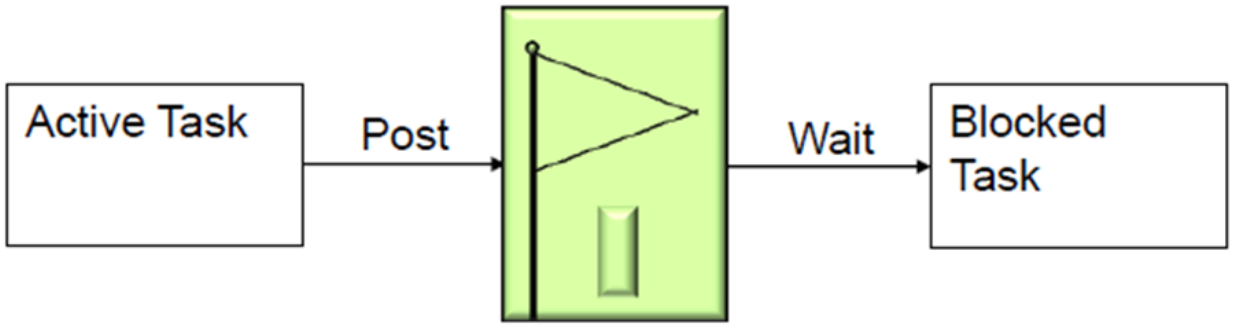
- Đồng bộ có thể được sử dụng để giải quyết:
  - Loại trừ lẫn nhau (Mutual Exclusion)
  - Luồng điều khiển (Control Flow)
  - Luồng dữ liệu (Data Flow)
- Cơ chế đồng bộ bao gồm:
  - Hàng đợi thông báo (Message queue)
  - Semaphore
  - Mutex
  - Event
- Cơ chế đồng bộ chính xác phụ thuộc vào vấn đề đồng bộ hóa đang được giải quyết

Loại trừ lẫn nhau

- Vấn đề: nhiều tác vụ có thể "đồng thời" cần truy cập vào cùng một tài nguyên
  - Tài nguyên có thể là mã, dữ liệu, thiết bị ngoại vi, v.v.
  - Cần cho phép tài nguyên được chia sẻ chỉ có thể truy cập độc quyền đối với một tác vụ tại một thời điểm
- Làm thế nào?
  - Chỉ cho phép một tác vụ khóa (lock) tài nguyên và các tác vụ còn lại phải đợi tài nguyên được mở khóa (unlock)
  - Các cơ chế phổ biến: lock/unlock, mutex, semaphore

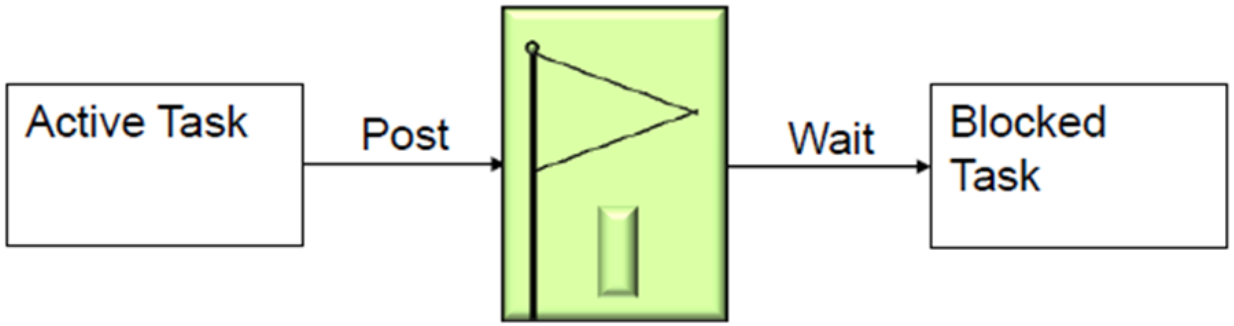
Đồng bộ luồng điều khiển

- Vấn đề: một tác vụ hoặc ISR có thể cần khôi phục thực thi một hoặc nhiều tác vụ khác, để các tác vụ thực thi theo thứ tự do ứng dụng kiểm soát
  - Loại trừ lẫn nhau được sử dụng để ngăn tác vụ khác chạy, trong khi luồng điều khiển được sử dụng để cho phép tác vụ khác chạy, thường là các tác vụ cụ thể
- Làm thế nào?
  - Các cơ chế phổ biến: post/wait, signal, event



Đồng bộ luồng dữ liệu

- Vấn đề: một tác vụ hoặc ISR có thể cần gửi một lượng dữ liệu đến một hoặc nhiều tác vụ cụ thể khác, để dữ liệu có thể được xử lý theo yêu cầu của ứng dụng chỉ định
- Làm thế nào?
  - Có thể được thực hiện gián tiếp thông qua đồng bộ hóa luồng điều khiển
  - Các cơ chế phổ biến: queue, signal, post/wait



ĐỒNG BỘ TÁC VỤ

- Giới thiệu về đồng bộ tác vụ
- **Semaphore trong FreeRTOS**
- Mutex trong FreeRTOS

Semaphore

- Semaphore được dùng để:
  - Điều khiển truy cập tới một tài nguyên chia sẻ (loại trừ lẫn nhau – Mutex)
  - Báo hiệu một sự kiện (event) xảy ra
  - Cho phép hai tác vụ đồng bộ hóa hoạt động của chúng
- Ý tưởng cơ bản:
  - Một semaphore chứa một số lượng token. Code cần nhận được một token để tiếp tục thực thi
  - Nếu tất cả token của semaphore đã được sử dụng, tác vụ yêu cầu sẽ bị đình chỉ (suspended) cho đến khi một số token được giải phóng bởi các chủ sở hữu hiện tại của chúng

Semaphore

- Semaphore làm việc như thế nào?
  - Một semaphore có:
    - Counter: Số lượng truy cập đồng thời tối đa
    - Queue: Cho các tác vụ chờ truy cập
  - Nếu một tác vụ yêu cầu (đợi - wait) một semaphore
    - Nếu counter> 0, thì (1) counter giảm đi 1 và (2) tác vụ nhận semaphore và tiến hành công việc của nó
    - Ngược lại, tác vụ bị chặn (Block) và đưa vào queue
  - Nếu một tác vụ giải phóng (đăng - post) một semaphore
    - Nếu có tác vụ trong hàng đợi semaphore, thì tác vụ thích hợp sẽ được chuẩn bị sẵn, theo chính sách xếp hàng
    - Ngược lại, counter được tăng thêm 1
- Có 2 loại semaphore: **Binary semaphore, Couting semaphore**

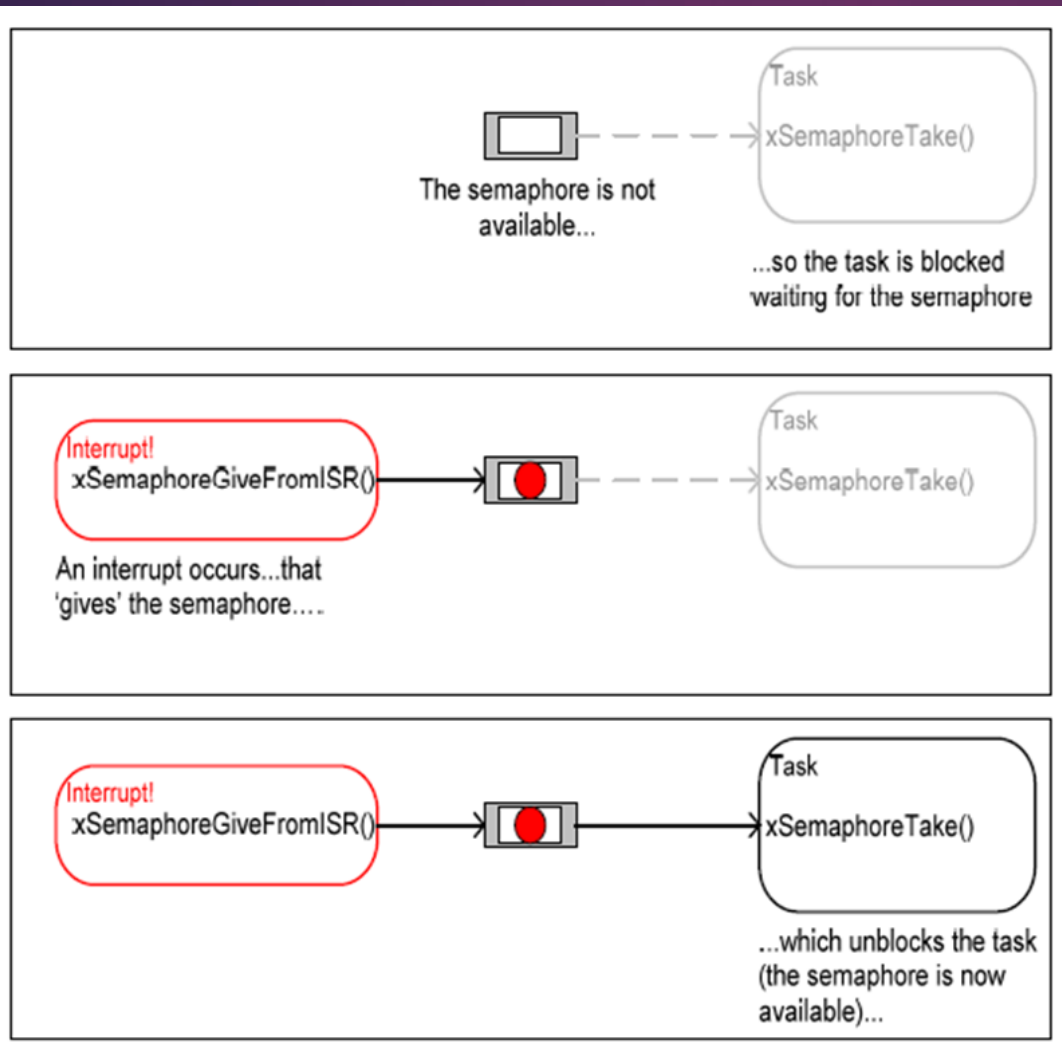
Binary Semaphore

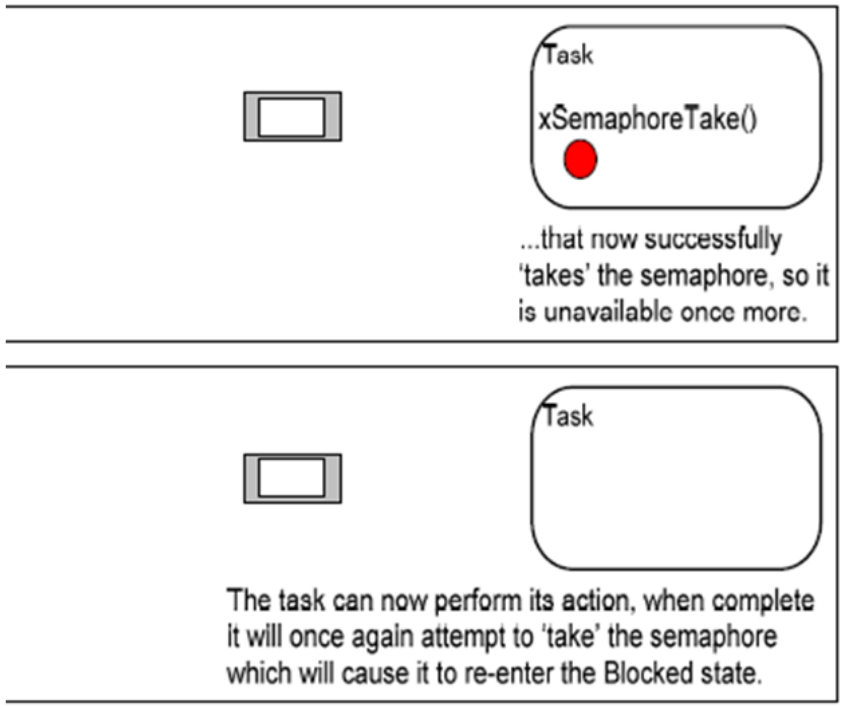
- Semaphore với counter = 1, sử dụng cho loại trừ lẫn nhau và đồng bộ hóa
- Cho mục đích đồng bộ hóa, một semaphore nhị phân có thể được hiểu như một queue chỉ có một phần tử dữ liệu (item)
  - Hàng đợi chỉ có thể trống hoặc đầy (do đó gọi là nhị phân)
  - Các tác vụ sử dụng hàng đợi không quan tâm hàng đợi chứa gì, chỉ muốn biết hàng đợi trống hay đầy
  - Nếu có nhiều hơn một tác vụ khóa (lock) trên cùng một semaphore, thì tác vụ có mức ưu tiên cao nhất sẽ là tác vụ được unlock vào lần semaphore khả dụng tiếp theo (semaphore đầy)

Binary Semaphore và Interrupt

- Cách tốt nhất để xử lý các sự kiện phức tạp do ngắt kích hoạt là không thực hiện mã trong ISR
  - Tạo một tác vụ đang chặn (block) trên semaphore nhị phân
  - Khi ngắt xảy ra, ISR chỉ đặt (give) semaphore và thoát
  - Tác vụ hiện tại có thể được lên lịch giống như bất kỳ tác vụ nào khác
    - Không cần phải lo lắng về việc lồng các ngắt (nesting interrupt) và mức độ ưu tiên của ngắt
- Điều này được gọi là Deferred Interrupt Processing

Binary Semaphore và Interrupt





Binary Semaphore

- Tạo semaphore nhị phân:

```
SemaphoreHandle_t xSemaphoreCreateBinary(void);
```

- Hàm tạo semaphore nhị phân:
  - Semaphore được tạo ở trạng thái “empty” (rỗng)
  - Một semaphore nhị phân không cần được trả (give) lại sau khi lấy (take), do đó, đồng bộ hóa tác vụ có thể được thực hiện bởi một tác vụ/ngắt liên tục 'cấp' (give) semaphore trong khi một tác vụ khác liên tục 'lấy‘ (take) semaphore
  - Semaphore nhị phân được gán cho các biến kiểu SemaphoreHandle_t và có thể được sử dụng trong bất kỳ hàm API nào nhận tham số kiểu này

Binary Semaphore

- Cấp (give) một semaphore nhị phân:

```
xSemaphoreGiveFromISR(
    SemaphoreHandle_t xSemaphore,
    signed BaseType_t *pxHigherPriorityTaskWoken)
```

- Cấp một semaphore nhị phân:
  - Có thể sử dụng từ một ISR
  - **`xSemaphore`**: semaphore handler của semaphore đang được phát hành
  - **`pxHigherPriorityTaskWoken:`** đặt thành pdTRUE nếu việc cấp semaphore dẫn đến tác vụ có mức độ ưu tiên cao hơn unlock, dẫn đến chuyển đổi ngữ cảnh

Binary Semaphore

- Reset (take) một semaphore nhị phân:

```
xSemaphoreTake(xSemaphoreHandle xSemaphore,
               portTickType xBlockTime)
```

- Reset (take) một semaphore nhị phân:
  - **`xSemaphore`**: semaphore handler của semaphore đang được phát hành
  - **`xBlockTime`** thời gian (tính bằng Tick) để đợi semaphore trở nên khả dụng
    - Bằng 0 có thể được sử dụng để thăm dò trạng thái của semaphore

Binary Semaphore

- Ví dụ:

```
vSemaphoreHandle binary_sem; //Global handler
void one_sec_isr(void){ // an ISR
  xSemaphoreGiveFromISR(binary_sem, NULL);
}
void sem_task(void *p){
  while(1)
    if(xSemaphoreTake(binary_sem,999999)) puts("Tick!");
}

int main(void){
    vSemaphoreCreateBinary(binary_sem);
    xTaskCreate(sem_task, (signed char*)) "t1", 2048,
          NULL, 1, NULL);
    vTaskStartScheduler();
    return 0;
}
```

Counting Semaphore

- Thường được sử dụng cho hai việc:
  - Đếm sự kiện:
    - Một trình xử lý sự kiện (event handler) sẽ 'cung cấp' (give) một semaphore mỗi khi một sự kiện xảy ra và một tác vụ xử lý (handler task) sẽ 'lấy' (take) một semaphore mỗi khi nó xử lý một sự kiện
  - Quản lý tài nguyên:
    - Giá trị đếm cho biết số lượng tài nguyên khả dụng
    - Để có được một tài nguyên, một tác vụ phải lấy (take) một semaphore
    - Khi một tác vụ kết thúc với tài nguyên, nó 'trả lại' (give) semaphore đó
- ```
  SemaphoreHandle_t xSemaphoreCreateCounting(
        UBaseType_t uxMaxCount,
        UBaseType_t uxInitialCount)
  ```
  ```

Counting Semaphore

- Ví dụ (1/2):

```
xSemaphoreHandle count_sem; //Global Handler

int main(void){
  /**parameter for uxMaxCount, uxInitialCount/
  count_sem = xSemaphoreCreateCounting(2, 2);

  /*Create tasks with priority 1 for both users*/
   xTaskCreate(task1, (signed char*)) "t1",
      1024, NULL, 1, NULL);
   xTaskCreate(task2, (signed char*)) "t2",
       1024, NULL, 1, NULL);

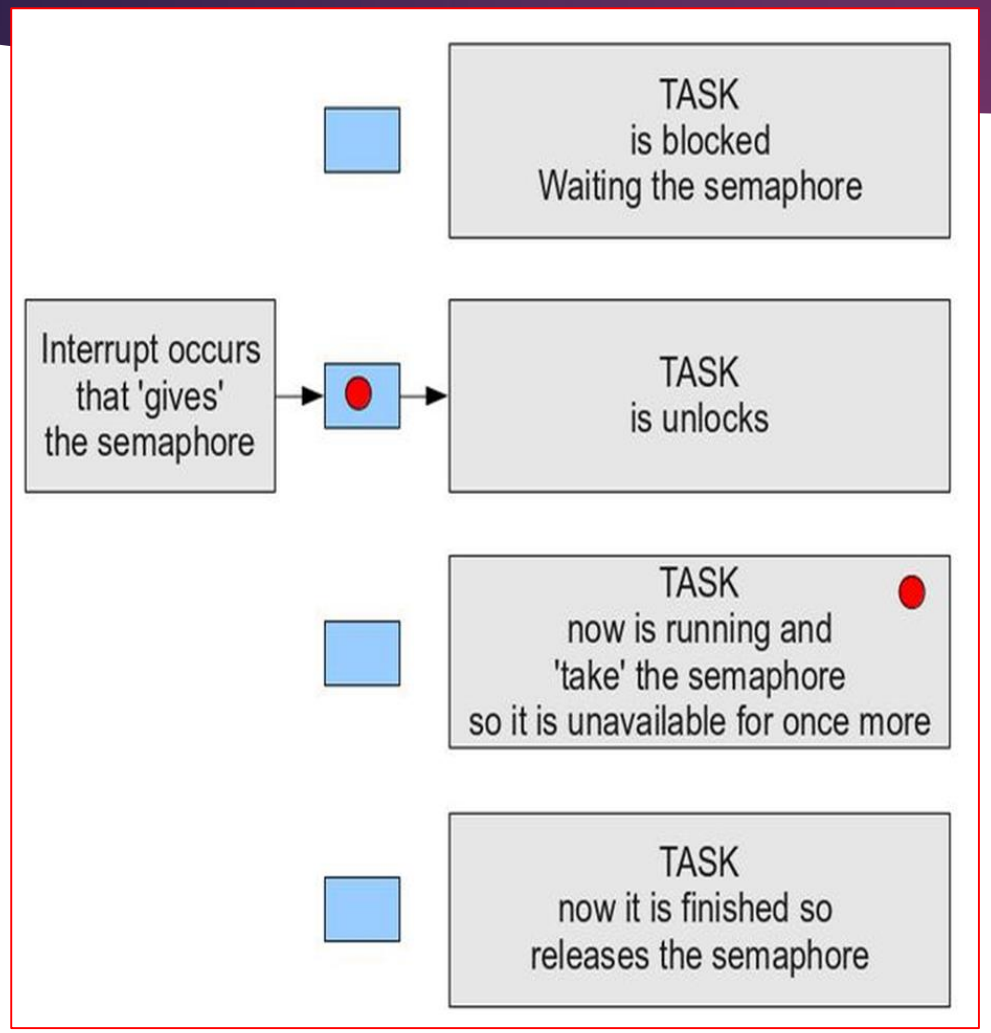
   vTaskStartScheduler();
   return 0;
}
```

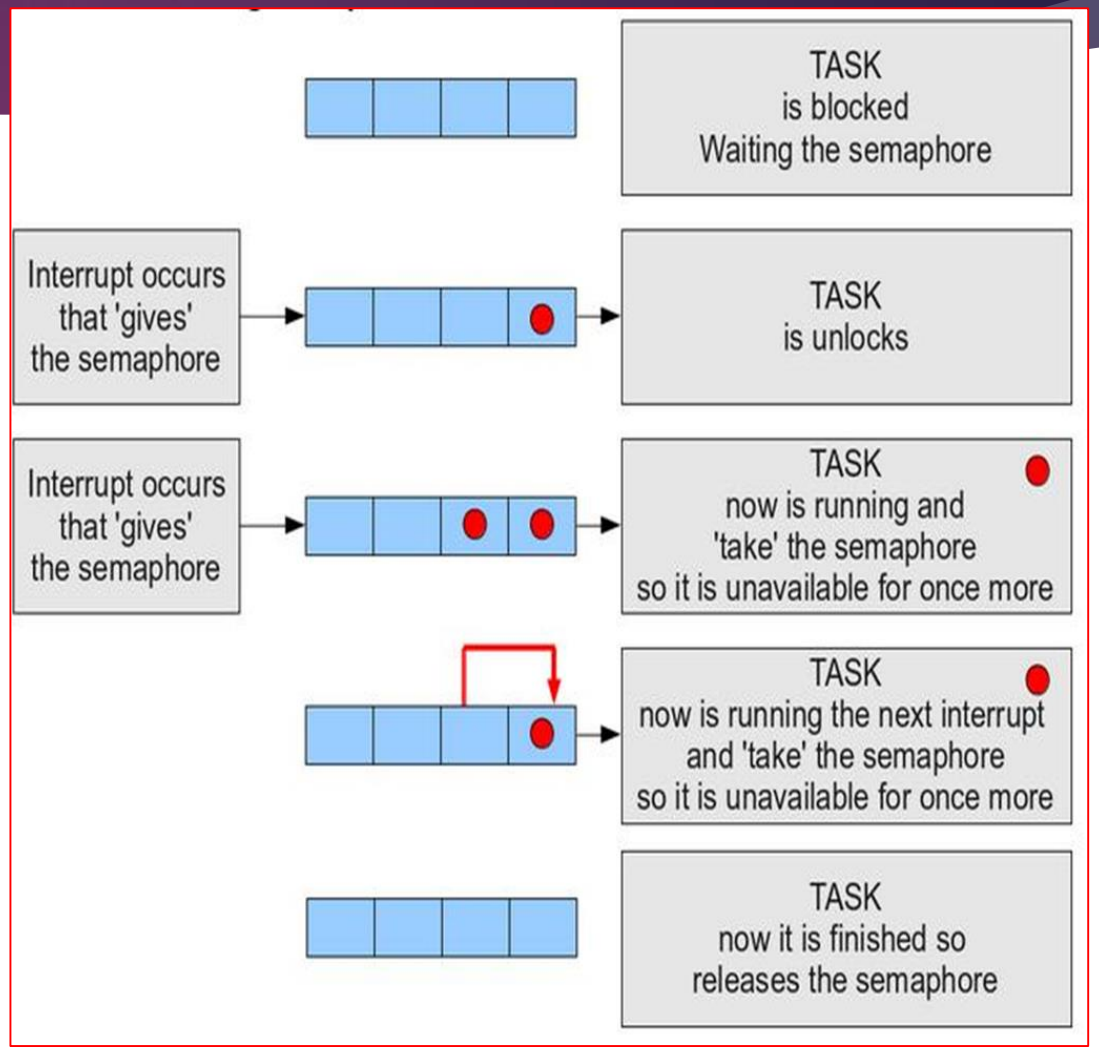
Counting Semaphore

- Ví dụ (2/2):

```
void task1(void *p){
  while(1){
    if(xSemaphoreTake(count_sem, portMAX_DELAY)){
       xSemaphoreGive(count_sem);
    }
    vTaskDelay(3000);
  }
}
void task2(void *p){
  while(1){
    if(xSemaphoreTake(count_sem), portMAX_DELAY){
       xSemaphoreGive(count_sem);
    }
    vTaskDelay(3000);
  }
}
```

So sánh: Binary Semaphore >< Counting Semaphore





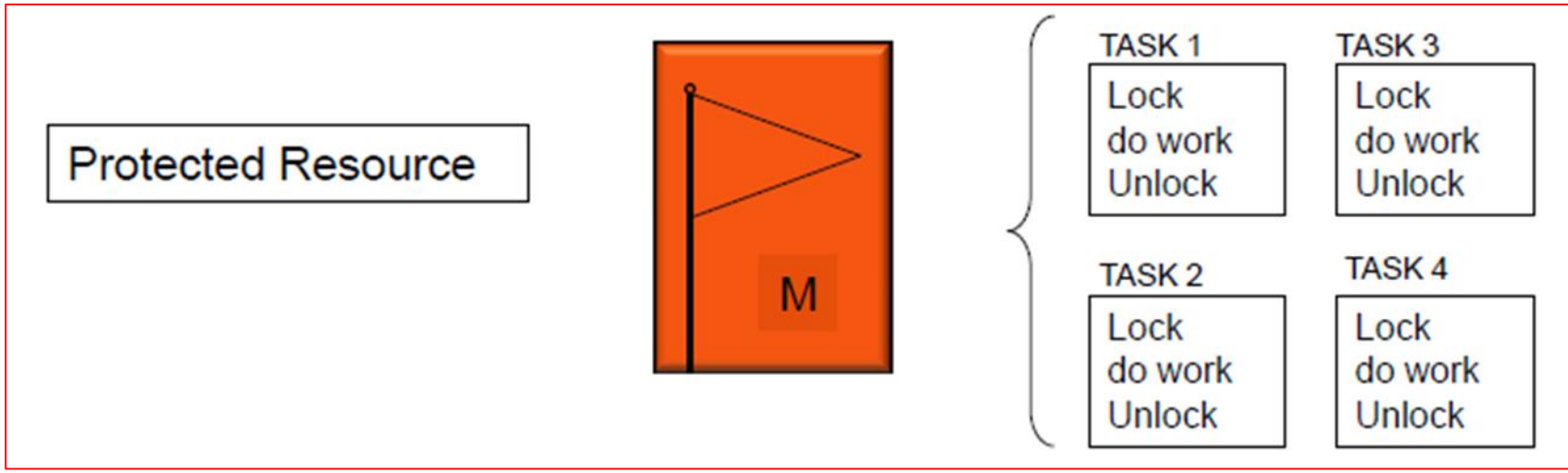
ĐỒNG BỘ TÁC VỤ

- Giới thiệu về đồng bộ tác vụ
- Semaphore trong FreeRTOS
- **Mutex trong FreeRTOS**

Mutex

- Được sử dụng cho việc loại trừ lẫn nhau, để tại một thời điểm chỉ một tác vụ sử dụng tài nguyên chia sẻ, ví dụ: file, dữ liệu, thiết bị, …
  - Để truy cập tài nguyên chia sẻ, một tác vụ khóa mutex liên kết với tài nguyên
  - Tác vụ sở hữu mutex cho đến khi nó unlock mutex



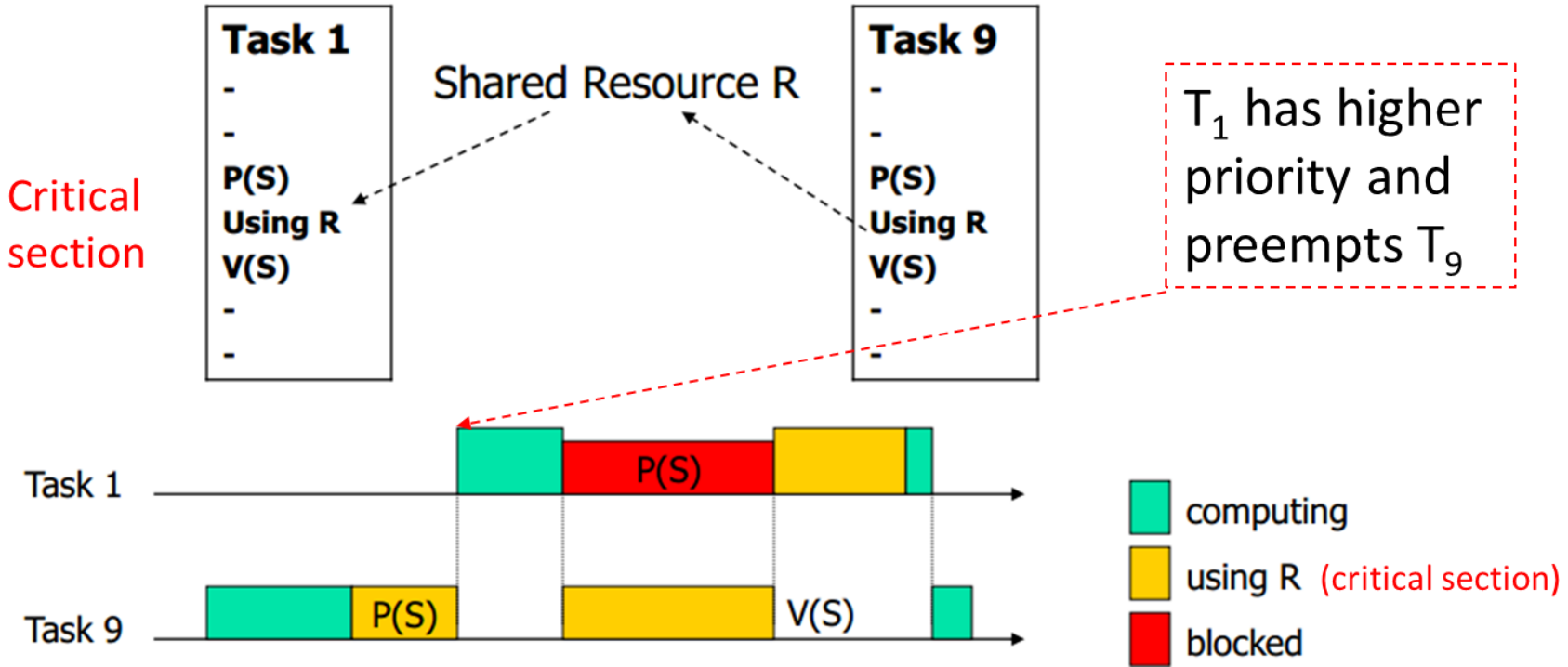


Mutex

- Mutex đóng vai trò như một token sử dụng để bảo vệ tài nguyên
  - Khi một tác vụ muốn truy cập tài nguyên, trước tiên nó phải lấy (take) token
  - Khi tác vụ kết thúc với tài nguyên, nó phải 'trả lại' (give) token - cho phép các tác vụ khác có cơ hội truy cập vào cùng một tài nguyên
- Mutex có thể khiến tác vụ có mức độ ưu tiên cao phải chờ đợi tác vụ có mức độ ưu tiên thấp hơn
  - Thậm chí tệ hơn, một tác vụ có mức độ ưu tiên trung bình có thể đang chạy và khiến cho tác vụ có mức độ ưu tiên cao không đáp ứng được thời hạn (deadline) của nó!
  - Vấn đề đảo ngược ưu tiên (Priority inversion)

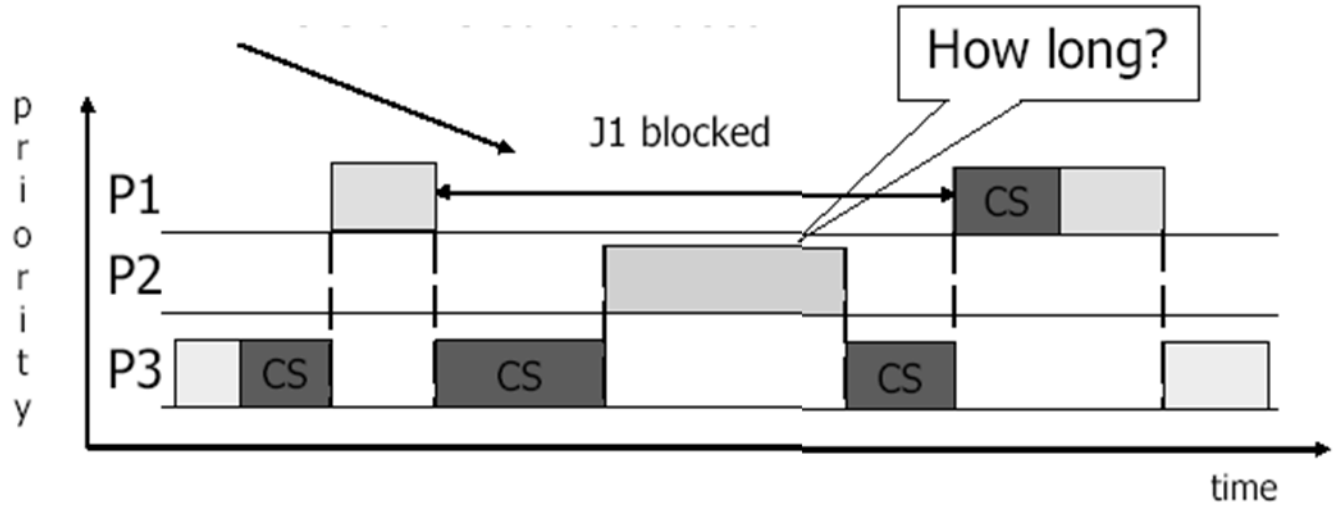
Priority Inversion:

- Trường hợp 1: Giả sử mức ưu tiên của T1 > mức ưu tiên của T9
  - Nếu T9 có quyền truy cập độc quyền, T1 phải đợi cho đến khi T9 giải phóng tài nguyên -> đảo ngược ưu tiên -> có thể nâng mức ưu tiên của T9



Priority Inversion:

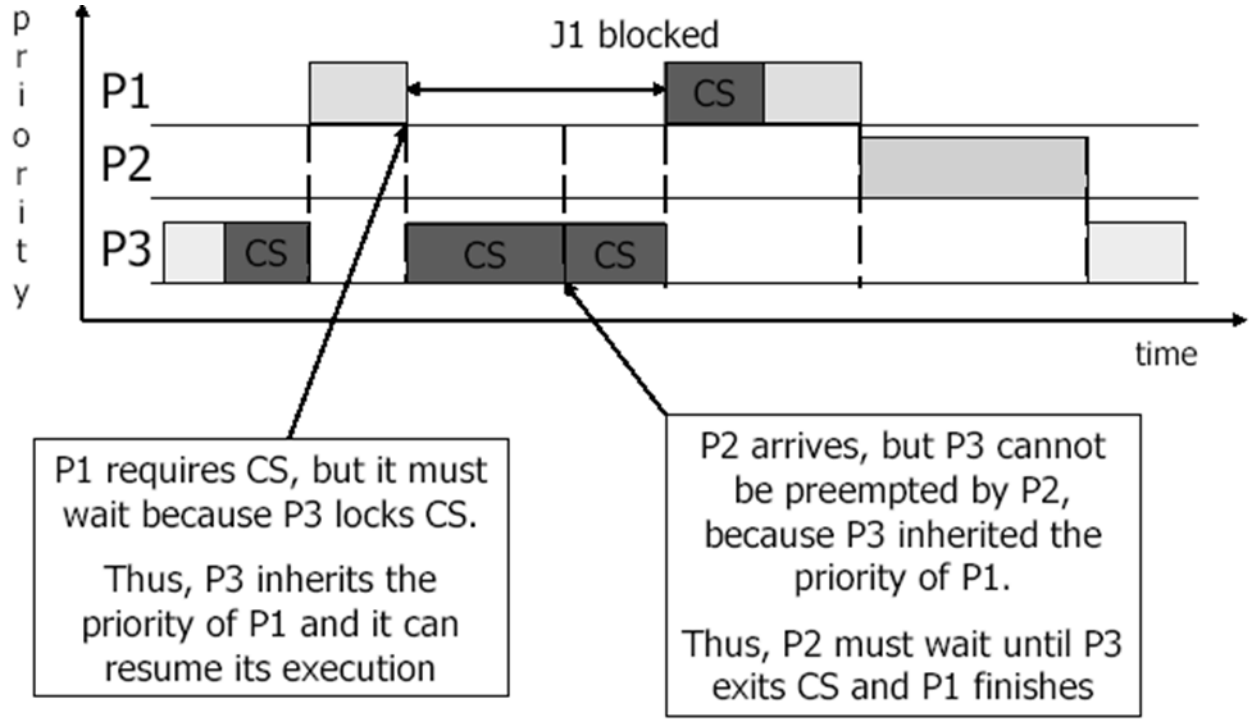
- Trường hợp 2: Một tác vụ có mức ưu tiên trung bình thay thế tác vụ có mức ưu tiên thấp hơn đang sử dụng tài nguyên chia sẻ mà trên đó tác vụ có mức ưu tiên cao hơn bị block.
  - Nếu tác vụ có mức ưu tiên cao hơn đã sẵn sàng để chạy, nhưng thay vào đó, tác vụ có mức độ ưu tiên trung bình hiện đang chạy, thì sẽ xảy ra đảo ngược ưu tiên



Priority Inversion:

Giải quyết:

- Kế thừa ưu tiên (Priority inheritance)
  - Nếu tác vụ có mức độ ưu tiên cao block trong khi cố gắng lấy mutex (token) hiện đang được giữ bởi tác vụ có mức độ ưu tiên thấp hơn, thì mức độ ưu tiên của tác vụ giữ token tạm thời được nâng lên mức độ ưu tiên của tác vụ block



Mutex: Ví dụ (1/3)

```
include <semphr.h>
SemaphoreHandle_t  gatekeeper = 0; //Global handler
void user_1(void *p){
  while(1){
    if(xSemaphoreTake(gatekeeper, 100)){
      Serial.println("User 1 got access");
      // enter critical section
      vTaskDelay(200); //stay in C.S. for 200 ticks
      xSemaphoreGive(gatekeeper);
      // release semaphore, exit critical section  }
    else{
      Serial.println("User 1 cannot access in 1000 ms");
    }
    vTaskDelay(100); // or do other works
    // Without delay, user 1 will get key immediately// }
}
```

Mutex: Ví dụ (2/3)

```
void user_2(void *p){
  while(1){
    if(xSemaphoreTake(gatekeeper, 100)){
      Serial.println("User 2 got access");
      //critical section
      xSemaphoreGive(gatekeeper);
      //release semaphore, exit critical section }
    else{//fail to get the semaphore
      Serial.println("User 2 cannot access in 1000 ms");
    }
    vTaskDelay(100); // or do other works
    // Without delay, user 2 will get key immediately
    after releasing the key //
  }
}
```

Mutex: Ví dụ (3/3)

```
void setup(){
    Serial.begin(9600);
    gatekeeper = xSemaphoreCreateMutex();
    // Create tasks with priority 1 for both users //
    xTaskCreate(user_1, (const portCHAR*)"t1", 128,
        NULL, 1, NULL);
    xTaskCreate(user_2, (const portCHAR*)"t2", 128,
        NULL, 2, NULL);
    Serial.println("test");
    vTaskStartScheduler();
}

void loop()
{
  ...
}
```

Bài tập thực hành: TH5 – Đồng bộ tác vụ

- TH5.1: Sử dụng Semaphore nhị phân
  - https://microcontrollerslab.com/freertos-binary-semaphore-tasks-interrupt-synchronization-u-arduino/
- TH5.2: Sử dụng Semaphore đếm
  - https://microcontrollerslab.com/freertos-counting-semaphore-examples-arduino/
- TH5.3: Sử dụng Mutex
  - https://microcontrollerslab.com/arduino-freertos-mutex-tutorial-priority-inversion-priority-inheritance/

Nội dung

- Đồng bộ tác vụ
- Truyền thông liên tác vụ

Truyền thông liên tác vụ

- RTOS cho phép phát triển các hệ thống nhúng phức tạp. Bằng cách sử dụng các tác vụ (luồng) độc lập, mỗi tác vụ có ngữ cảnh riêng, ta có thể triển khai các chương trình có hành vi đa nhiệm bằng cách sử dụng một CPU duy nhất.
- Truyền thông giữa các tác vụ (giao tiếp giữa các tác vụ) là một khía cạnh quan trọng khi thiết kế một ứng dụng nhúng bằng RTOS.
- Có thể nói rằng giao tiếp giữa các tác vụ được thực hiện trong các hệ thống nhúng thời gian thực dựa trên một hoặc sự kết hợp của các cơ chế sau:
  - Chia sẻ bộ nhớ (memory sharing)
  - Cơ chế tín hiệu (signaling mechanism)
  - Truyền thông điệp (message passing)

Memory sharing

- Điều đầu tiên nghĩ đến như một cơ chế truyền thông tin giữa các tác vụ khác nhau là sử dụng vị trí bộ nhớ dùng chung.
- Điều quan trọng là bộ nhớ dùng chung được bảo vệ bằng mutex hoặc semaphore.
- Một ví dụ rất cơ bản về việc sử dụng vị trí bộ nhớ chia sẻ là một biến toàn cục (Global variable)
  - Mặc dù không có gì ngăn cản ta sử dụng một biến như vậy, nhưng nó không được khuyến khích vì có nhiều cách phức tạp hơn sẵn có như là phương tiện giao tiếp giữa các tác vụ.

Signaling mechanism

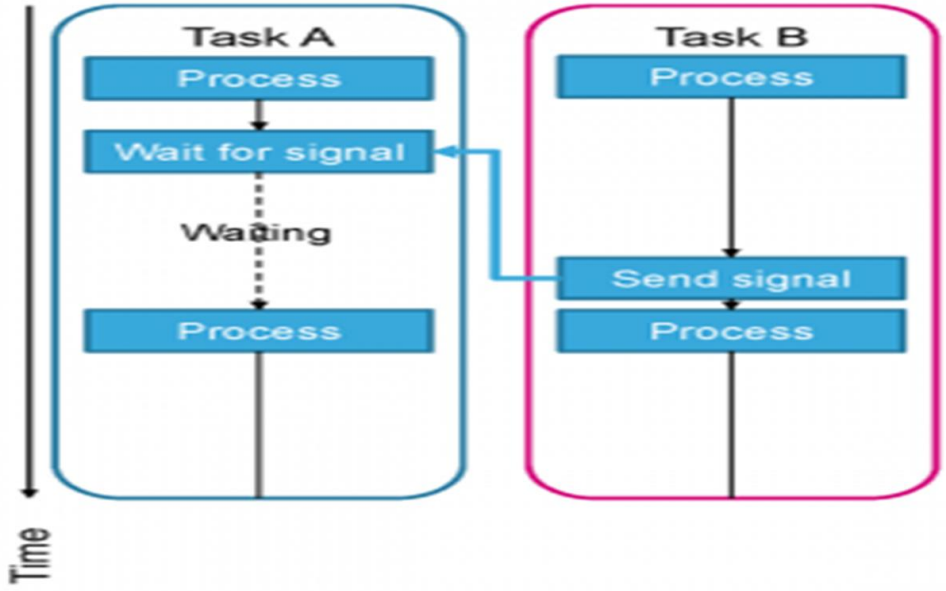
- Là một dạng truyền thông cơ bản.
- Chỉ ra sự xuất hiện của một sự kiện và có thể sử dụng cho mục đích đồng bộ hóa
- Ta đã đề cập đến một cơ chế như vậy được gọi là semaphore trong phần trước.
- Hai cơ chế báo hiệu bổ sung được sử dụng rộng rãi trong các hệ điều hành thời gian thực:
  - Cờ sự kiện (event flags)
  - Cờ tác vụ (task flags)
- Vì không có quy ước đặt tên nghiêm ngặt, có thể thấy các cơ chế này có tên hơi khác nhau trong các bản phân phối RTOS khác nhau, nhưng chức năng mà chúng phục vụ là khá giống nhau.

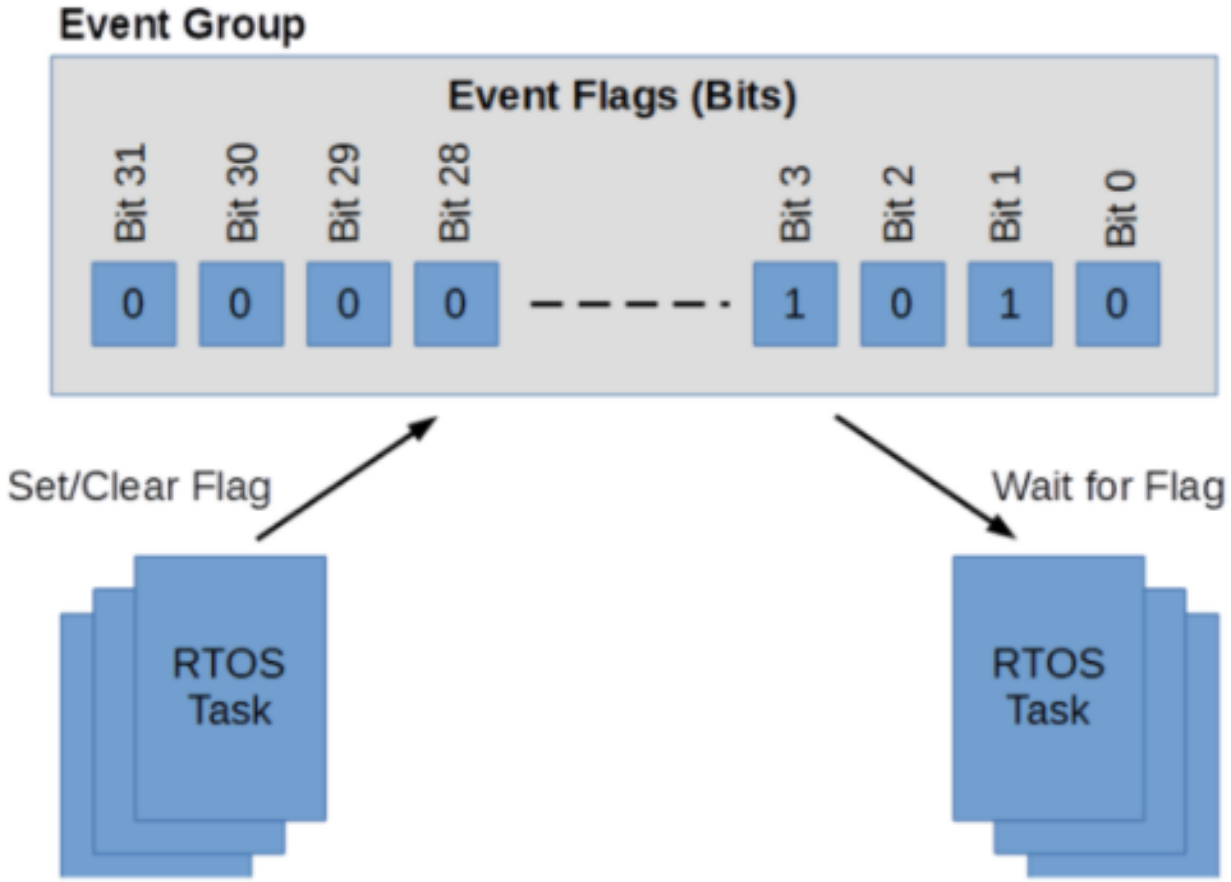
Signaling mechanism

- **Cờ sự kiện (event flags)**
  - Là các bit sử dụng để mã hóa thông tin cụ thể
  - Được sử dụng để đồng bộ hóa và truyền thông các tác vụ.
  - Nhóm các cờ sự kiện riêng lẻ được gọi là nhóm sự kiện (**event group**) hay tín hiệu (**signal**).
  - Cờ sự kiện có thể được sử dụng bởi các tác vụ và theo chương trình phục vụ ngắt (ISR). Một cờ sự kiện đơn lẻ (hoặc một nhóm) có thể được truy cập bởi nhiều tác vụ khác nhau.
  - Các hoạt động phổ biến nhất có thể được thực hiện trên cờ sự kiện là:
    - Create/Delete event flags
    - Set/Clear event flags
    - Read a flag's value
    - Wait on a flage to take a specific value

Signaling mechanism

- **Cờ sự kiện (event flags)**





Signaling mechanism



- **Cờ sự kiện (event flags) trong FreeRTOS: Event group**
  - Event group: chủ yếu có hai thuật ngữ quan trọng là cờ sự kiện (event flag) và bit sự kiện (event bit).
  - Event flag là một giá trị boolean là ‘0’ hoặc ‘1’. Các giá trị boolean này biểu thị sự xuất hiện hoặc không xảy ra của một sự kiện.
  - Trong FreeRTOS, Biến EventBits_t được sử dụng để lưu trữ trạng thái của cờ sự kiện.
    - Nếu giá trị của EventBits_t là 1 có nghĩa là một sự kiện liên quan đến bit cụ thể này xảy ra. Ngược lại, nếu EventBits_t = 0, sự kiện đã không xảy ra.
    - Ví dụ, nếu giá trị event group là 0x92 (=1001 0010) nghĩa là chỉ sự kiện 1, 4, 7 xảy ra

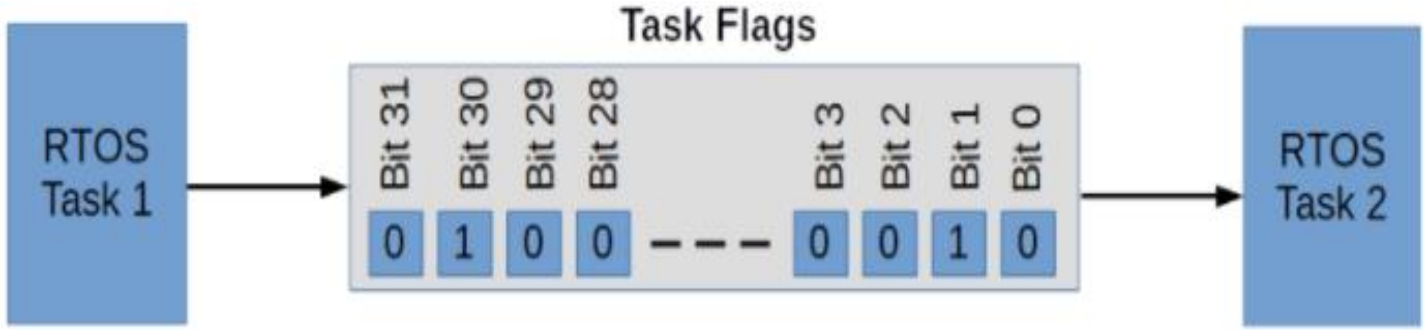
Signaling mechanism

- **Cờ sự kiện (event flags) trong FreeRTOS: Event group**
  - Thiết đặt kích cỡ event group: định nghĩa số bít cờ trong một event group đơn lẻ. Ta có thể lựa chọn bằng cách sử dụng hằng số configUSE_16_BIT_TICKS của FreeRTOSConfig.h.
    - Nếu configUSE_16_BIT_TICKS = 1, event group chứa 8 flags.
    - Ngược lại nếu configUSE_16_BIT_TICKS = 0, event group bao gồm 24 flags.

- **Các hàm API**:
  - xEventGroupCreate
  - xEventGroupWaitBits
  - xEventGroupSetBits
  - xEventGroupSetBitsFromISR
  - xEventGroupClearBits
  - xEventGroupClearBitsFromISR
  - xEventGroupGetBits
  - xEventGroupGetBitsFromISR
  - vEventGroupDelete

Signaling mechanism

- **Cờ tác vụ (Task flags)**
  - Là một dạng đặc biệt của event flag
  - Trong khi cờ sự kiện có thể được truy cập bởi tất cả tác vụ, cờ tác vụ được sử dụng để thông báo (notification) đến một tác vụ nhận đơn lẻ (single receiving task)
  - Các hoạt động phổ biến nhất có thể được thực hiện trên cờ tác vụ là:
    - Set/Clear flags của tác vụ cụ thể
    - Wait on a flage to take a specific value
  - RTOS: thread flags
  - FreeRTOS: Task notifications



Message passing

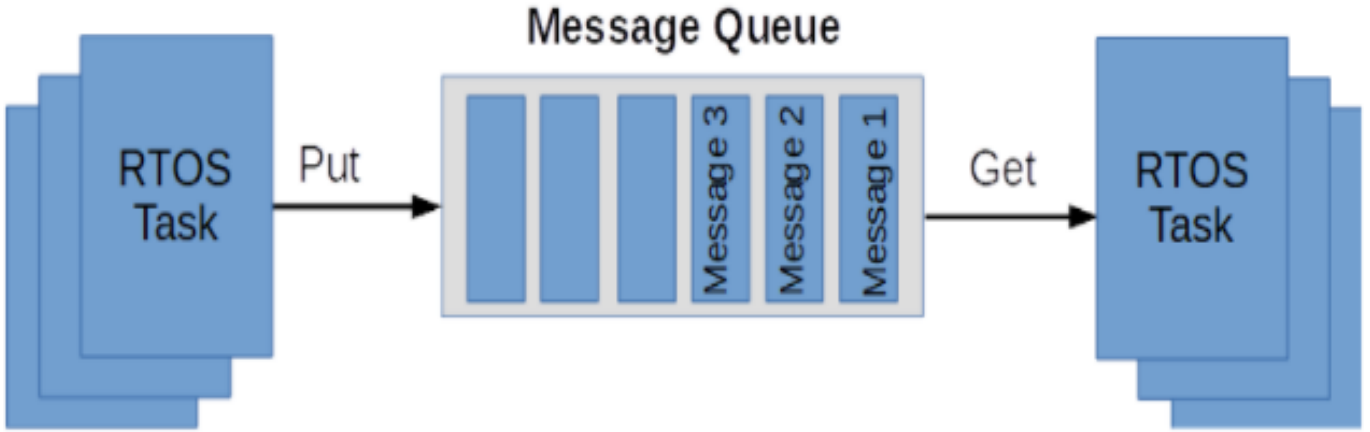
- Nhìn chung, có thể xác định hai kiểu:
  - Truyền thông điệp trực tiếp - Người gửi và người nhận thông điệp được xác định rõ ràng.
    - Ví dụ: FreeRTOS phổ biến có Stream&Message Buffer như là cơ sở ban đầu để truyền thông điệp trực tiếp giữa một tác vụ ghi đơn lẻ và một tác vụ đọc duy nhất.
  - Truyền thông điệp gián tiếp – Các hông điệp được đặt trong các cấu trúc như hàng đợi thông điệp (message queue) hoặc hộp thư (mailbox) và nhiều tác vụ có quyền đọc/ghi.

Message passing

- Message queue
  - Là bộ đệm dữ liệu với số lượng mục nhập (entry) hữu hạn. Mỗi entry có thể chứa dữ liệu có kích thước nhất định (ví dụ: 32bits).
  - Có thể được sử dụng để truyền dữ liệu giữa các tác vụ và giữa các chương trình phục vụ ngắt ISR với các tác vụ.
  - Được triển khai dưới dạng bộ đệm FIFO an toàn cho luồng (thread-safe FIFO buffers).
  - Các hành động cụ thể được định nghĩa trong RTOS trong trường hợp một tác vụ cố gắng ghi vào hàng đợi thông báo đầy (full) hoặc cố gắng đọc một hàng đợi thông báo trống (empty).

Message passing

- Message queue
  - Các hoạt động phổ biến nhất có thể được thực hiện là:
    - Create/delete a message queue
    - Get the number of messages currently stored in the queue
    - Put a message into the queue
    - Get a message from the queue



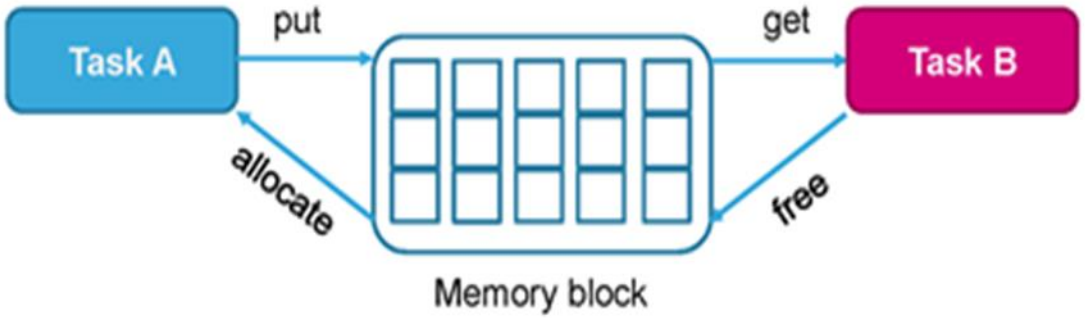
Message passing

- Message queue
  - Ví dụ: Sử dụng hàng đợi tin nhắn để đệm dữ liệu
    - Có hai tác vụ đang hoạt động ở các tốc độ khác nhau. Ta có thể sử dụng hàng đợi thông báo làm bộ đệm nếu một trong các tác vụ tạo ra một loạt các mẫu dữ liệu và tác vụ kia phải xử lý từng mẫu dữ liệu riêng lẻ với tốc độ cố định.



Message passing

- Mailbox
  - Có thể lưu trữ một dữ liệu có kích thước cụ thể (ví dụ: biến 32-bit) và có thể được triển khai dưới dạng hàng đợi một mục nhập (single-entry queue).
  - Một mailbox duy nhất có thể được truy cập bởi nhiều tác vụ.
  - Trong một số bản phân phối RTOS, mailbox có thể có nhiều hơn một mục nhập, điều này làm cho chúng rất giống với một hàng đợi thông báo (message queue) trong phần trước.
  - Hoạt động điển hình có thể được thực hiện trên mailbox là:
    - Create/delete a mailbox
    - Write to a mailbox
    - Read a mailbox



Bài tập thực hành: TH6 – Truyền thông liên tác vụ

- TH6.1: Sử dụng Mailbox
  - https://microcontrollerslab.com/create-mailbox-with-queues-using-freertos-arduino/
  - Hoặc: https://www.youtube.com/watch?v=rqSAKKi5WsQ
- TH6.2: Sử dụng Event Group:
  - https://microcontrollerslab.com/freertos-event-groups-tasks-synchronization-example-arduino/
- TH6.3: Sử dụng Queue
  - Arduino: Hai chân 8,9 có hai đèn led, ban đầu đèn số 8 bật, đèn số 9 tắt.
  - Khởi tạo 2 task, một task gửi dữ liệu và một task nhận dữ liệu
  - Task gửi tạo ra một giá trị mỗi giây và thêm giá trị này vào cuối hàng đợi
  - Task nhận sẽ đọc một giá trị từ đầu hàng đợi cứ sau 500 ms, nếu không có giá trị, nó sẽ đợi trong 1 giây. Nếu giá trị nhận được lớn hơn hoặc bằng 4 và nhỏ hơn hoặc bằng 10 thì đèn số 8 sẽ tắt và đèn số 9 sẽ bật. Ngược lại nếu giá trị nhận được nhỏ hơn 4 và lớn hơn 10 thì đền số 8 sẽ bật và đèn số 9 sẽ tắt.

Bài tập thực hành: TH6 – Truyền thông liên tác vụ

- TH6.4: Sử dụng Queue
  - Arduino, LCD, ADC: https://microcontrollerslab.com/arduino-freertos-queues-create-read-write-examples/

Q & A